

day 13

📌 01. Document & JSON Schema Validation

المفهوم الأساسي 💡 📌

بالرغم من أن MongoDB هي **Schema-less**، إلا أنها توفر من بداية الإصدار 3.2 (وتطور في 3.6 لـ **JSON Schema**) إمكانية فرض قواعد صحة البيانات (**Validation Rules**) على الـ Collections لضمان وجود حقول معينة وأنواع بيانات محددة أثناء عمليات الـ `insert` والـ `update`.

1. إنشاء Collection بـ JSON Schema (`db.createCollection`) 🟢

تُحدد القواعد باستخدام الخيار `validator` وداخلها `jsonSchema$`.

JavaScript

```
db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["firstName", "lastName"], // 🔥 حقول إجبارية
      additionalProperties: false,          // 🚫 يمنع إضافة أي حقول غير
      // معرف (إلا _id)
      properties: {
        _id: {}, // 🔑 يجب تعريفه دائماً عند استخدام
      },
      additionalProperties: false
      firstName: { bsonType: "string" },
      lastName: { bsonType: "string" },
      age: { bsonType: "number", maximum: 15.0 }
    }
  }
});
```

2. تعديل الـ Validation لـ Collection حالية (`collMod`) 🟡

لتعديل القواعد على Collection موجودة مسبقاً نستخدم الأمر `runCommand` مع `collMod`.

JavaScript

```

db.runCommand({
  collMod: "students",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["firstName", "lastName"],
      additionalProperties: false,
      properties: {
        _id: {},
        firstName: { bsonType: "string" },
        lastName: { bsonType: "string" },
        age: { bsonType: "number", maximum: 15.0 }
      }
    }
  },
  validationAction: "error",    // Options: "error" (Default) | "warn"
  validationLevel: "moderate"  // Options: "strict" (Default) | "moderate"
  | "off"
});

```

Validation (Validation Options)

Option	Value	Description
validationAction	"error"	يمنع كتابة/تعديل البيانات ويرمي Error فوراً عند مخالفة القواعد (Default).
	"warn"	يسمح بالكتابة ولكن يسجل تنبيه المخالفة في الـ Log.
validationLevel	"strict"	يطبق القواعد على جميع عمليات الإدخال والتعديل للكل (Default).
	"moderate"	يطبق القواعد على الإدخال الجديد، والتعديلات على المستندات التي كانت صحيحة مسبقاً فقط .
	"off"	تعطيل الـ Validation تماماً.

02. Update Operators Reference

عند تحديث البيانات نستخدم صيغة: `db.collection.updateMany(condition, updateCommand, options)`

- **\$set** : لتعديل قيمة حقل أو إضافة حقل جديد لو مش موجود.
- **\$unset** : لحذف حقل بالكامل من المستند.
- **\$rename** : لإعادة تسمية حقل معين.
- **\$inc** : لزيادة أو تقليل قيمة رقمية (مثل المرتب).
- **\$push / \$pull / \$pop** : (إضافة، حذف عنصر محدد، أو حذف أول/آخر عنصر) Arrays للتعامل مع الـ.

- **Positional Operator** \$: Filter بشرط وجوده في الـ Array داخل الـ Index لتعديل عنصر غير معروف الـ :

حل وإجابات MongoDB Lab 2 بالتفصيل

افتحي الـ MongoDB Shell (mongosh) وطبقي الكويريز التالية:

1 كويريز الاستعلام والفلتر (Queries)

a- **instructors collection** عرض جميع المستندات في

JavaScript

```
db.instructors.find()
```

b- **4000** (فقط **salary** و **firstName** إظهار) عرض المدرسين اللي مرتبهم أكبر من

JavaScript

```
db.instructors.find(
  { salary: { $gt: 4000 } },
  { firstName: 1, salary: 1, _id: 0 }
)
```

c- **25** :عرض المدرسين بعمر أقل من أو يساوي

JavaScript

```
db.instructors.find({ age: { $lte: 25 } })
```

d- **10** أو **14 Mansoura** عرض مدرسين مدينة (**firstName**, **address**, **salary**):

JavaScript

```
db.instructors.find(
  { "address.city": "mansoura", "address.street": { $in: [10, 14] } },
  { firstName: 1, address: 1, salary: 1, _id: 0 }
)
```

e- **(both of them)** معاً **jquery** و **js** عرض المدرسين اللي عندهم كورسات

JavaScript

```
db.instructors.find({ courses: { $all: ["js", "jquery"] } })
```

f- عرض عدد الكورسات لكل مدرس مع الاسم الأول:

JavaScript

```
db.instructors.aggregate([
  {
    $project: {
      firstName: 1,
      numberOfCourses: { $size: { $ifNull: ["$courses", []] } },
      _id: 0
    }
  }
])
```

g- ثم **firstName** تصاعدياً بالـ **lastName** و **firstName** ترتيب المدرسين الذين لديهم **g- age** والـ **FullName** وعرض **lastName** تنازلياً بالـ

JavaScript

```
db.instructors.find(
  { firstName: { $exists: true }, lastName: { $exists: true } },
  { FullName: 1, age: 1, _id: 0 }
).sort({ firstName: 1, lastName: -1 })
```

 **BONUS:** إنشاء **collection** باسم **instructorsData** جديدة بخطوة **g** بنتيجة خطوة

JavaScript

```
db.instructors.aggregate([
  { $match: { firstName: { $exists: true }, lastName: { $exists: true } } },
  { $sort: { firstName: 1, lastName: -1 } },
  { $project: { FullName: 1, age: 1, _id: 0 } },
  { $out: "instructorsData" }
])
```

h- "mohammed" الخاص بهم على **fullName** البحث عن المدرسين الذين يحتوي الـ

JavaScript

```
db.instructors.find({ fullName: { $regex: /mohammed/i } })
```

2 كويريز التعديل والحذف (Update & Delete)

i- ولديه 5 كورسات فقط في القائمة "ebtesam" حذف المدرس باسم:

JavaScript

```
db.instructors.deleteOne({ firstName: "ebtesam", courses: { $size: 5 } })
```

j- لجميع المدرسين true وقيمتها active باسم property إضافة:

JavaScript

```
db.instructors.updateMany({}, { $set: { active: true } })
```

k- بدون معرفة الـ "mazen mohammed" للمدرس "jquery" إلى "EF" تغيير كورس Index):

JavaScript

```
db.instructors.updateOne(
  { firstName: "mazen", lastName: "mohammed", courses: "EF" },
  { $set: { "courses.$": "jquery" } }
)
```

l- "noha hesham" للمدرسة "jquery" إضافة كورس:

JavaScript

```
db.instructors.updateOne(
  { firstName: "noha", lastName: "hesham" },
  { $push: { courses: "jquery" } }
)
```

m- "ahmed mohammed" للمدرس courses المسماة property حذف الـ:

JavaScript

```
db.instructors.updateOne(
  { firstName: "ahmed", lastName: "mohammed" },
```

```
{ $unset: { courses: "" } }  
)
```

n- تقليل المرتب بمقدار 500 لكل من لديهم 3 كورسات بس في القائمة

JavaScript

```
db.instructors.updateMany(  
  { courses: { $size: 3 } },  
  { $inc: { salary: -500 } }  
)
```

o- للكل **fullAddress** إلى **address** تغيير اسم الفيلد

JavaScript

```
db.instructors.updateMany({}, { $rename: { address: "fullAddress" } })
```

p- ليصبح **noha hesham 20** تغيير رقم الشارع لـ

JavaScript

```
// لأننا قمنا بتغيير الاسم في الخطوة fullAddress تم استخدام  
db.instructors.updateOne(  
  { firstName: "noha", lastName: "hesham" },  
  { $set: { "fullAddress.street": 20 } }  
)
```

? أسئلة إنترفيو متوقعة (Interview Q&A)

? Q1: What is the difference between `$elemMatch` and `$all` in MongoDB?

الإجابة:

- تحتوي على جميع العناصر المحددة بغض النظر عن ترتيبها أو شرط متداخل Array تُستخدم للتحقق من أن الـ `$all`.
- يحق أكثر Object لضمان أن نفس العنصر/الـ `Array of Objects` تُستخدم عند التصفية في `$elemMatch`. من شرط داخلي في نفس الوقت.

? Q2: What is the purpose of `{ upsert: true }` in update operations?

الإجابة: تعني (Update + Insert). إذا وجد المونجو مستند يطابق الشرط سيقوم بتحديثه، أما إذا لم يجد مستند مطابق فسيقوم بإنشاء مستند جديد بالبيانات الممررة.

Q3: How do you view existing schema validation rules on a collection?

الإجابة: باستخدام دالة `db.getCollectionInfos({ name: "collectionName" })`.